

Instructor Handout 5: Addressing Modes, CPU Organization, and Bus Structures

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

Instructor Copy — Not for Student Distribution

5.1 Before You Teach This Lecture

- **Mano, *Computer System Architecture*, 3rd ed., Sec. 8-5 “Addressing Modes,” pp. 256–266** (indexed, page-cited). Read the book’s own worked numerical example (Fig. 8-7) — a second, book-native worked instance to fall back on if a student’s question needs different numbers than the ones below.
- **Mano, Sec. 8-2 “General Register Organization,” pp. 242–246** (indexed). The section Fig. 8-2 (“Registers with common ALU”) actually comes from — read it once so you can explain SELA/SELB/SELD confidently.
- **Mano, Fig. 5-4, p. 129** (indexed at the figure level). The real single-bus/common-bus-system figure the single-bus datapath below is styled on.
- **Hamacher, *Computer Organization*, Ch. 7** — *not yet page-indexed* for this course. Cite at chapter level only with students.

5.2 Addressing Modes

Open with the motivating example: a compiler translating $A[i] = A[i] + 1$; inside a loop generates something like `ADD R1, X`, but the instruction format never says where `X` lives — it could be a constant, a fixed cell, a pointer, or a changing array slot. The **addressing mode** is what specifies which.

Definition (Addressing Mode). *An addressing mode is the rule, specified by the instruction, for computing the **effective address** (EA) — the actual memory location or register holding the operand — from the instruction’s address field.*

Walk the seven modes in this order, each motivated by the code idiom it implements, not as an isolated list to memorize:

- **Immediate** (`ADD R1, #5`): the address field *is* the operand — zero extra cycles, but the value must be known at compile time.

- **Direct** (ADD R1, 1000): $EA = A$, one memory read — but the address is baked in permanently.
- **Indirect** (ADD R1, (1000)): $M[A]$ holds another address, not the operand. If $A = 300$, $M[300] = 500$, $M[500] = 42$, then $EA = 500$ and the operand is 42 — **two** memory reads.
- **Register / register-indirect**: same two ideas, but the pointer’s home moves into a register. Register-indirect costs one memory read instead of indirect’s two, because reading a register is free of a bus transaction.
- **Displacement** ($EA = [\text{register}] + d$): this is how array indexing actually compiles — R_{base} holds $\&A[0]$, R_{index} holds i , and $EA = \&A[i]$ is recomputed every execution, which is exactly why direct addressing cannot serve this role.
- **Autoincrement/autodecrement**: register-indirect plus an automatic ± 1 side effect — saves a separate increment instruction per loop iteration.

Teaching note: Students routinely conflate indirect addressing (pointer in *memory*) with register-indirect (pointer in a *register*). Cold-call early: “which one is cheaper, and why?” Draw the indirect-addressing chain live, one box at a time (instruction $\rightarrow M[300] \rightarrow M[500] \rightarrow$ operand) rather than pre-drawn — the two-hop structure lands better built incrementally. For autoincrement, ask “what would this look like as two separate instructions?” before revealing the answer (a load, then a separate increment) — it motivates *why* the side-effect mode exists.

Example 5.2.1 (Computing the Effective Address). *LOAD R1, 20(R2), displacement addressing, $[R2] = 1000$: $EA = 1000 + 20 = 1020$; cost is one register read plus one memory read.*

Example 5.2.2 (A Second Instance: Autoincrement). *ADD R1, (R3)+, $[R3] = 500$, operand size 4 bytes. $EA = 500$ (before the increment); the operand is fetched from $M[500]$; then, as a side effect, $[R3] \leftarrow 504$. Worth emphasizing live: the increment happens after EA is used, not before — students sometimes assume pre-increment by analogy to $++i$.*

5.3 CPU Organization and Register-Transfer Notation

Addressing modes answer *where*; this section answers *how*. Sketch the CPU as a control unit, ALU, and register file on a short internal bus, with MAR/MDR as the gateway to memory — this is the establishing shot before either datapath below is built in detail.

Introduce RTL next: $R1 \leftarrow [R2]$ means “the contents of R2 transfer into R1”; $[X]$ always means *contents of X*; $M[EA]$ is the memory cell at EA ; a **control step** $T_1, T_2, \dots$ is one clock cycle’s worth of transfers.

Then pose the central design question: ADD R1, R2, R3 needs two operands at the ALU simultaneously and one result back. If everything shares one bus, only one value can be on it per cycle — two operands *cannot* arrive together. Two designs follow.

5.4 Single-Bus Organization

How to present it: draw the four top registers and the bus line first, with the taps, before saying anything about Y or the ALU. Then state the constraint out loud — only one value on the bus at a time — and *ask* where the ALU’s first operand waits while the second arrives, before drawing Y. Add the ALU, then Z, then the feedback loop last, narrating each addition as the answer to the question you just asked rather than as a pre-planned diagram.

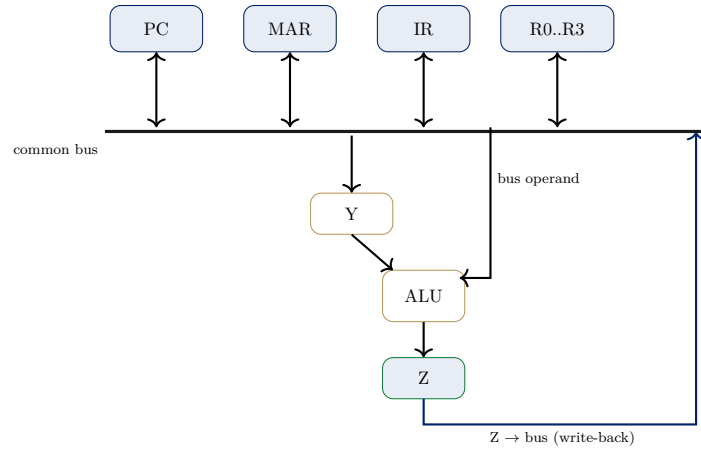


Figure 5.1: The single-bus CPU datapath — cf. Mano, Fig. 5-4, p.129; Y/Z are this course’s pedagogical simplification of the ALU operand path.

Teaching note: This derivation is the one students most often need a second pass on. A fast first pass through the 3-step trace, then a second, slower pass where the class supplies each T_i line themselves, works better than one careful pass. The recurring wrong instinct is to treat Y/Z as decorative complexity rather than a forced consequence of the one-bus constraint — the “where does it wait” question fixes this if asked *before* the registers are drawn.

Example 5.4.1 ($R1 \leftarrow R2+R3$ on a Single Bus). $T_1 : Y \leftarrow [R2]; \quad T_2 : Z \leftarrow [Y] + [R3]; \quad T_3 : R1 \leftarrow [Z]$. *Three control steps — the baseline for comparison below.*

Example 5.4.2 (A Second Instance: $R7 \leftarrow R0+R1$). *Same three steps, different registers — worth stating explicitly that register names never affect the step count, only the operation’s shape does.*

5.5 Multiple-Bus Organization

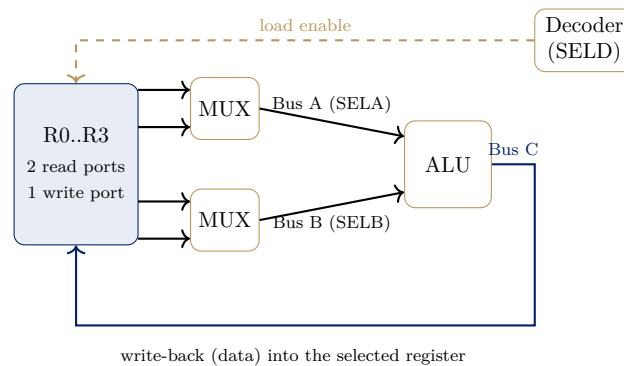


Figure 5.2: The three-bus CPU datapath — cf. Mano, Fig. 8-2, p.243 (“Registers with common ALU”).

How to present it: draw the register file first, explicitly narrating “two read ports, one write port” — that phrase is the whole design in miniature. Add the two MUXes and Bus A/Bus B next,

and say out loud that these are *separate physical wires*, which is why both operands can arrive in the same cycle (contrast explicitly with the single-bus board, where they cannot). Add the ALU and Bus C, then draw the decoder *last*, off to the side, with a visually distinct (dashed) arrow — the decoder does not carry data, only a load-enable signal, and drawing it last, separately, prevents students from reading it as part of the data path.

Teaching note: The common wrong instinct is attributing the speed gain to “a faster ALU” rather than parallel read paths — same ALU, same clock, different wiring; say this explicitly. Before revealing the diagram, ask “why not build a CPU with ten buses instead of three?” — most classes converge on “wiring cost” unprompted, which sets up the design-space-limits discussion that follows in the Notes.

Example 5.5.1 ($R1 \leftarrow R2+R3$ on Three Buses). $T_1 : R1 \leftarrow [R2] + [R3]$, with $A = [R2]$, $B = [R3]$, $C = \text{result}$ — one control step.

Example 5.5.2 (A Second Instance: $R2 \leftarrow R6-R7$). Same single-cycle structure, with the ALU set to subtract — worth doing live to pre-empt the assumption that the three-bus speed advantage is addition-specific.

5.6 Summary — Quick Reference

Addressing: EA from immediate (none), direct (A), indirect ($M[A]$), register-indirect ($[R]$), or displacement ($[R] + d$). **RTL:** $R \leftarrow [\cdot]$, $M[EA]$, control steps $T_1, T_2, \dots$. **Single-bus:** cheap, sequential, needs Y/Z, 3 steps for ADD. **Three-bus:** parallel reads via separate Bus A/B, write-back via Bus C decoupled from the decoder-driven SELD load-enable, 1 step for ADD.